Introduction to JavaScript Language

Ehsan Khamespanah

Spring 2026

Dynamic Behavior for Browsers

- HTML is used to specify the static structure of a web site
- There is a need for a programming language for the browser side
- It should be able to change HTML tags of the web site
- It also should be able to send request to web servers

JavaScript Language

- “The” language of the web browsers
- Widely used
- Various implementations freely available
- Useful for both web and non-web applications
- Complementing technologies: jQuery, Node.js, CouchDB

JavaScript Nature

- It’s dynamic
- It’s dynamically typed
- It’s functional and object-based
- It fails silently
- It’s deployed as source code
- It’s part of the web platform



I decided to learn

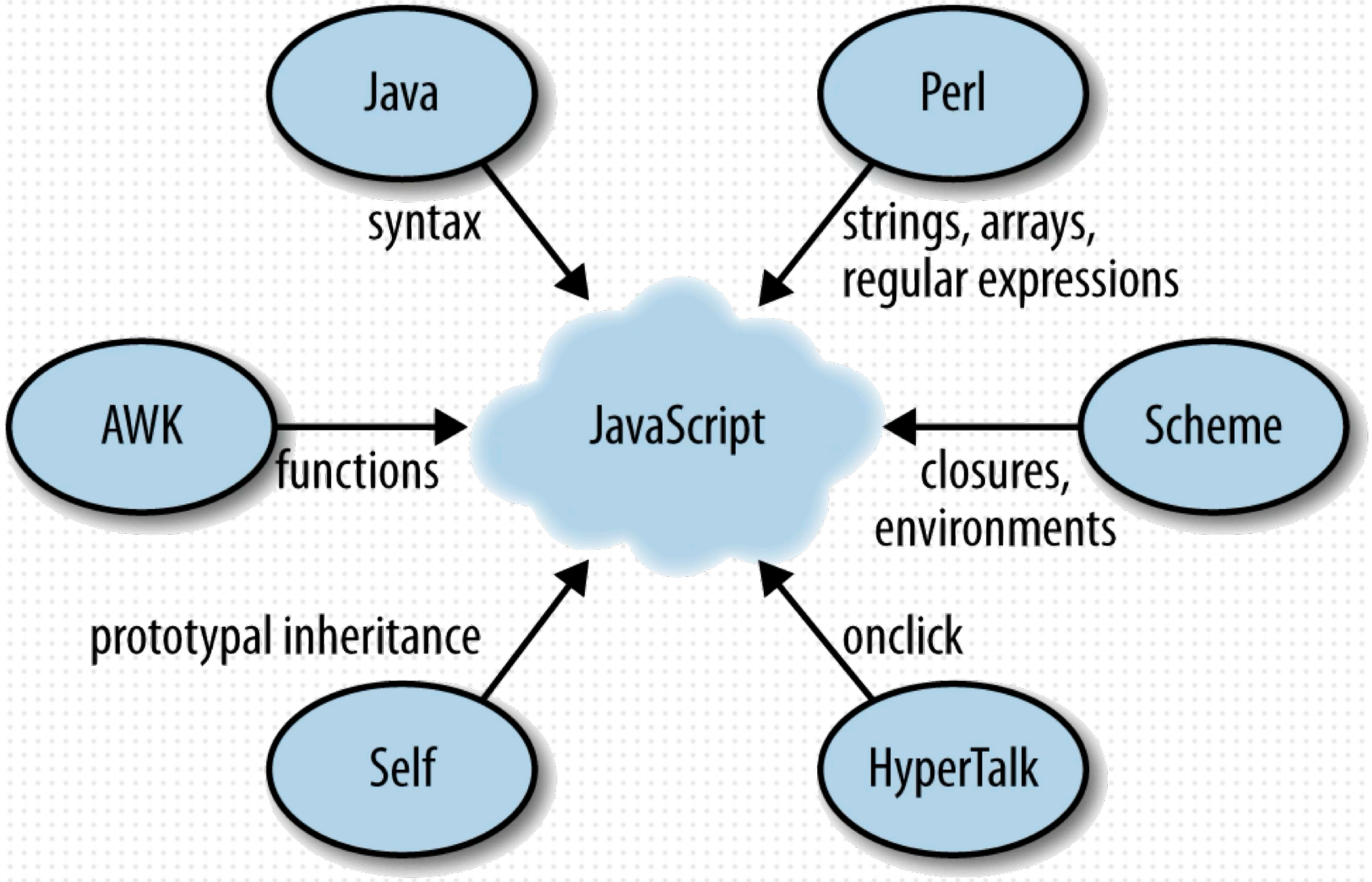
Javascript



```
[] + {} //'[object Object]'

{} + [] //0
```

Influences


Java
Perl
syntax
strings, arrays,
regular expressions
AWK
JavaScript
Scheme
functions
closures,
environments
prototypal inheritance
onclick
Self
HyperTalk

History

- 1995, Netscape, and Brendan Eich
- Decision on a scripting language with Java-like syntax
- Mocha → LiveScript → JavaScript
- Standardization by Ecma Intl.
- ECMAScript specification



Netscape

ecma INTERNATIONAL

Sun microsystems

Using JavaScript

- In-browser scripts using <script> tag
- Embedded scripting language (e.g., Chrome extensions, CouchDB queries, OpenOffice.org scripts)
- Using scripting engines (e.g., QtScript, javax.script, etc.)
- Using application platforms (e.g., ActionScript, Ubuntu Touch, etc.)
- Using Node.js runtime engine (mainly used for server-side programming)

Syntax

- Very similar to the widely used programming languages

```
var x; // declaring a variable
x= 3 + y; // assigning a value to the variable 'x'

foo(x, y); // calling function 'foo' with parameters 'x' and 'y'
obj.bar(3); // calling method 'bar' of object 'obj'

// A conditional statement
if (x===0){ // Is 'x' equal to zero?
  x = 123;
}

// Defining function 'baz' with parameters 'a' and "b'
function baz(a, b) {
  return a + b;
}
```

Type System

- Dynamically typed
- Implicit type conversion between most types
- Primitive types: Boolean, Number, String
- Everything else is an object (including arrays and functions)!

Boolean	`true, false`
Number	`1735, 1.65e-4, NaN, Infinity`
String	`‘abc’, “abc”`
Two special non-values	`null, undefined`

Type System

- Dynamically typed
- Implicit type conversion between most types
- Primitive types: Boolean, Number, String
- Everything else is an object (including arrays and functions)!

Plain Objects	`{ firstName: 'Jane', lastName: 'Doe' }`
Arrays	`[ 'apple', 'banana', 'cherry' ]`
Regular Expressions	`/^a+b+$/`
Functions	`function (x) { return 2 * x; }`

Object References and Comparison

```
> var obj1 = {};  // an empty object
> var obj2 = {};  // another empty object
> obj1 === obj2
false

> var obj3 = obj1;
> obj3 === obj1
true
```

```
> var prim1 = 123;
> var prim2 = 123;
> prim1 === prim2
true
```

Undefined and Null

```
> Number(null)
0
> 5 + null
5
```

```
> Number(undefined)
NaN
> 5 + undefined
NaN
```

```
// Is x a non-value?
if (x === undefined || x === null) {
    ...
}
```

```
// Is x falsy?
if (!x) {
    ...
}
```

WARNING

`false`, `0`, `NaN`, and `''` are also considered `false`.

Type Coercition

```
> '3' * '4'
12
```

```
> 3 + ' times'
'3 times'
```

```
var formData = { width: '100' };

// You think formData.width is a number
// and get unexpected results
var w = formData.width;
var outer = w + 20;

// You expect outer to be 120, but it's not
console.log(outer === 120);  // false
console.log(outer === '10020');  // true
```

```
var w = Number(formData.width);
```

Equality Operators: == vs ===

- JavaScript has two ways of determining whether two values are equal:
  - Strict equality (===) and strict inequality (!==) consider only values that have the same type to be qual
  - Normal equality (==) and inequality (!=) try to convert values of different types before comparing them

```
> 1 == true
true
> 0 == false
true
```

```
> undefined == null
true
```

Statements

- Variable declarations
- Assignments
- Usual control structures
  - if, while, do-while, for, break, continue, switch
  - Exception handling (throw, try-catch blocks)
  - Function invocation

Functions

```
//Function Declaration
function add(x, y) {
  return x + y;
}

//Function Expression
var add = function (x, y) { return x + y };
console. log(add (2, 3)); //5

//Named Function Expression
var fac = function me(n) {
  if (n> 0) {
    return n * me(n-1);
  } else {
    return 1;
  }
};
console.log(fac(3)); //6

//Function Constructor
var add = new Function('x', 'y', 'return x + y');
```

Basic Objects

```
var jane = {
  name: 'Jane'
  describe: function () {
    return 'Person named '+this.name; // (1)
  }, // (2)
};
```

```
> jane.name // get property 'name'
'Jane'

> jane.describe // get property 'describe'
[Function]

> jane.unknownProperty
undefined

> jane.age = 43
```

Basic Objects

```
> var obj = { foo: 'a', bar: 'b' };
> obj.foo = undefined;
> Object. keys(obj)
[ 'foo', 'bar' ]

> delete obj.foo
true
> Object. keys(obj)
[ 'bar' ]

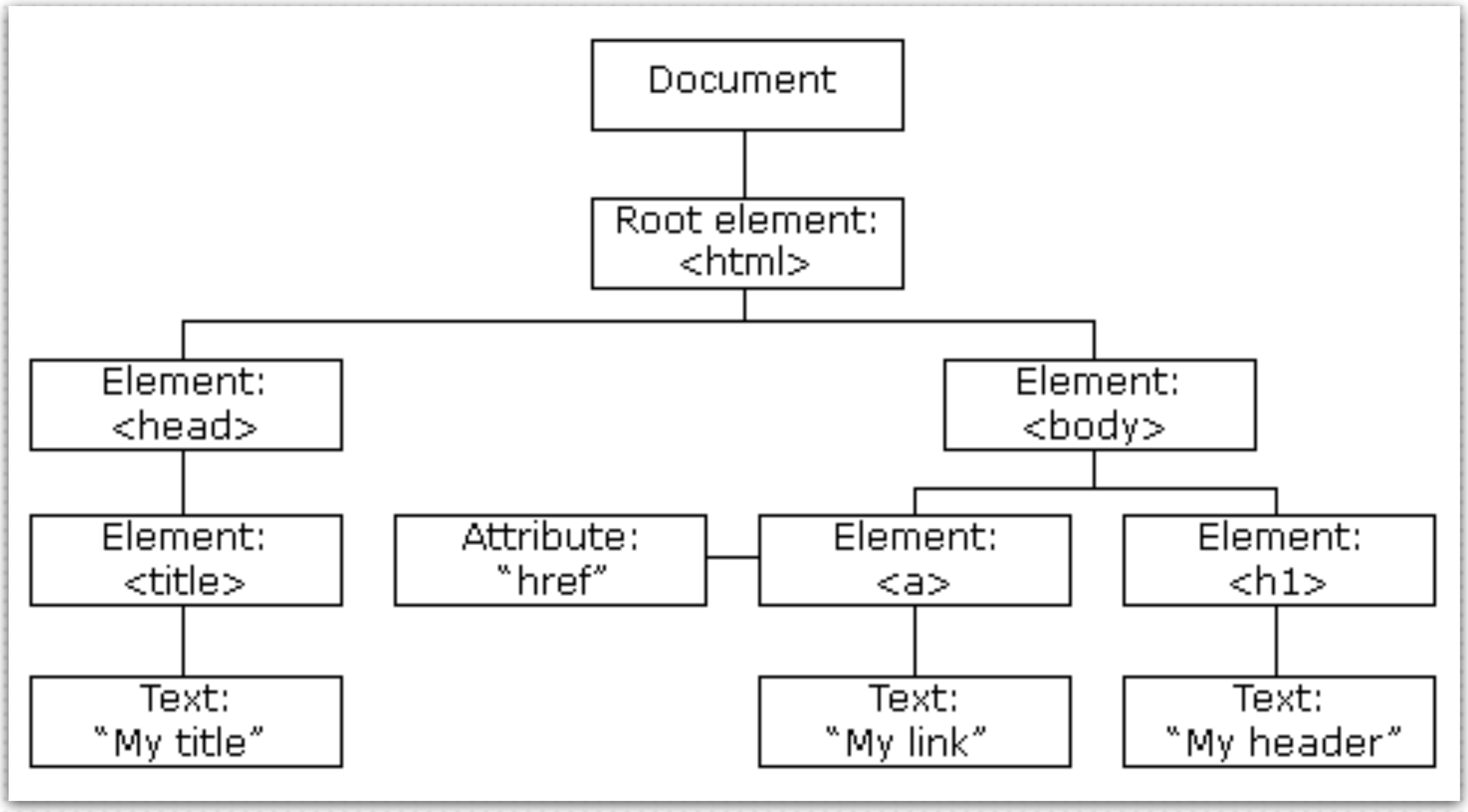
> var obj = { 'not an identifier': 123 };
> Object. keys(obj)
[ 'not an identifier' ]
> obj['not an identifier']
123

> var obj = { someProperty: 'abc' };
> obj['some' + 'Property']
'abc'
> var propKey = 'someProperty';
> obj[propKey]
'abc'

> var obj = { myMethod: function () { return true } };
> obj['myMethod']()
true
```

JavaScript HTML DOM

- Document Object Model



DOM API

```
<html>
<body>
  <p id="p1">Hello World! </p>
  <script>
    document.getElementById("p1").innerHTML = "New text!";
  </script>
</body>
</html>
```

DOM API

```
<html>
<body>
  <p id="p1">Hello World! </p>
  <script>
    document.getElementById("p1").innerHTML = "New text!";
  </script>
</body>
</html>
```

	Method
Finding	document.getElementById(*id*)
	document.getElementsByTagName(*name*)
	document.getElementsByClassName(*name*)

DOM API

```
<html>
<body>
  <p id="p1">Hello World! </p>
  <script>
    document.getElementById("p1").innerHTML = "New text!";
  </script>
</body>
</html>
```

Finding

Method
documer
documer
documer

Modify

Method
element.innerHTML = *new html content*
element.attribute = new value
element.setAttribute*(attribute, value)*
element.style.*property = new style*

DOM API

```
<html>
<body>
  <p id="p1">Hello World! </p>
  <script>
    document.getElementById("p1").innerHTML = "New text!";
  </script>
</body>
</html>
```

Finding

Method
documer
documer
documer

Modify

Method
element.
element.
element.
element.style.*property = new style*

Add/Delete Elements

Method
document.createElement(*element*)
document.removeChild(*element*)
document.appendChild(*element*)
document.replaceChild(*element*)
document.write(*text*)

Finding Objects

`document.body`	`document.lastModified`
`document.URL`	`document.title`
`document.forms`	`document.referrer`
`document.images`	`document.scripts`
`document.links`	`document.readyState`

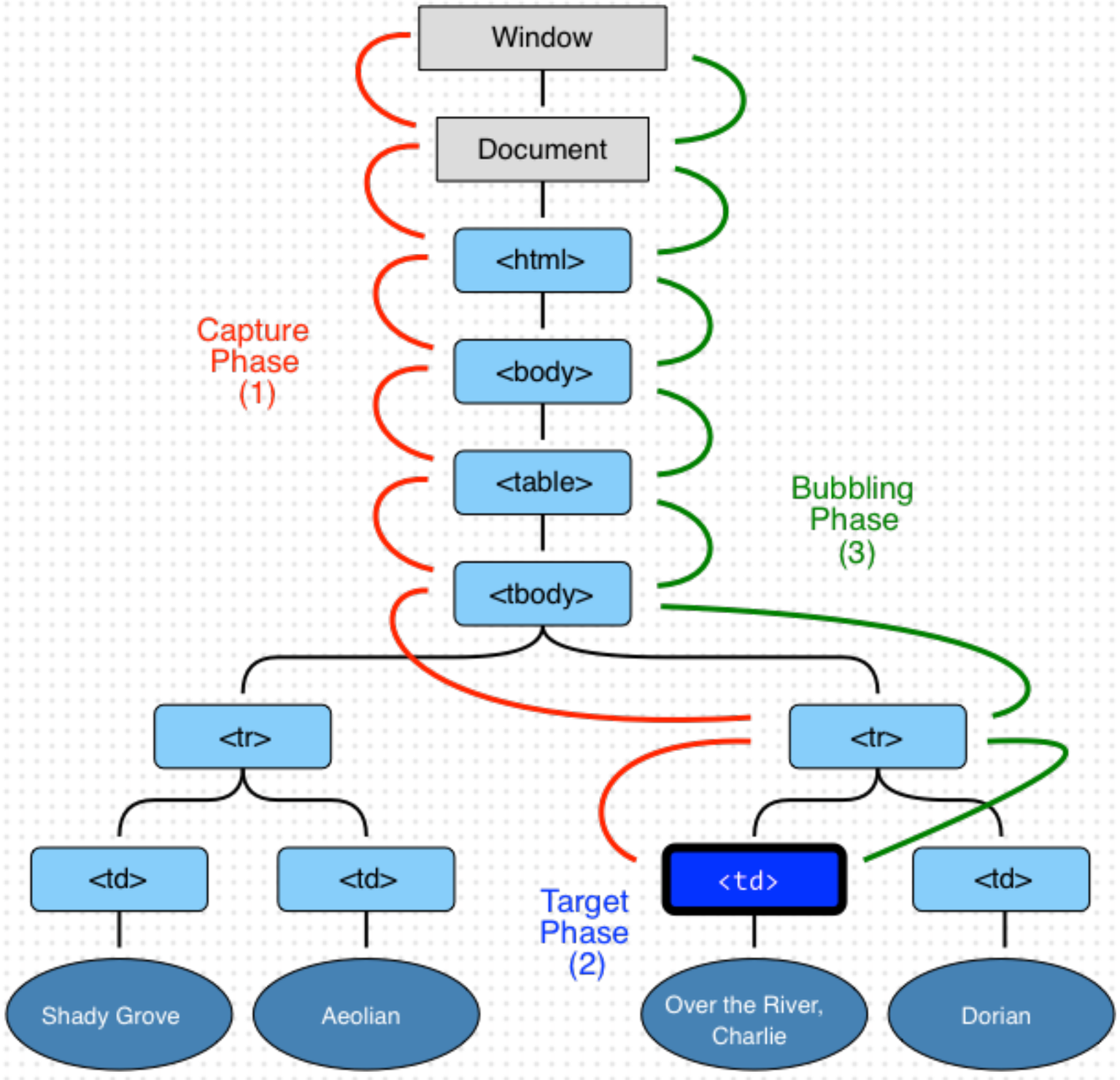
and, there are more…

Changing Styles

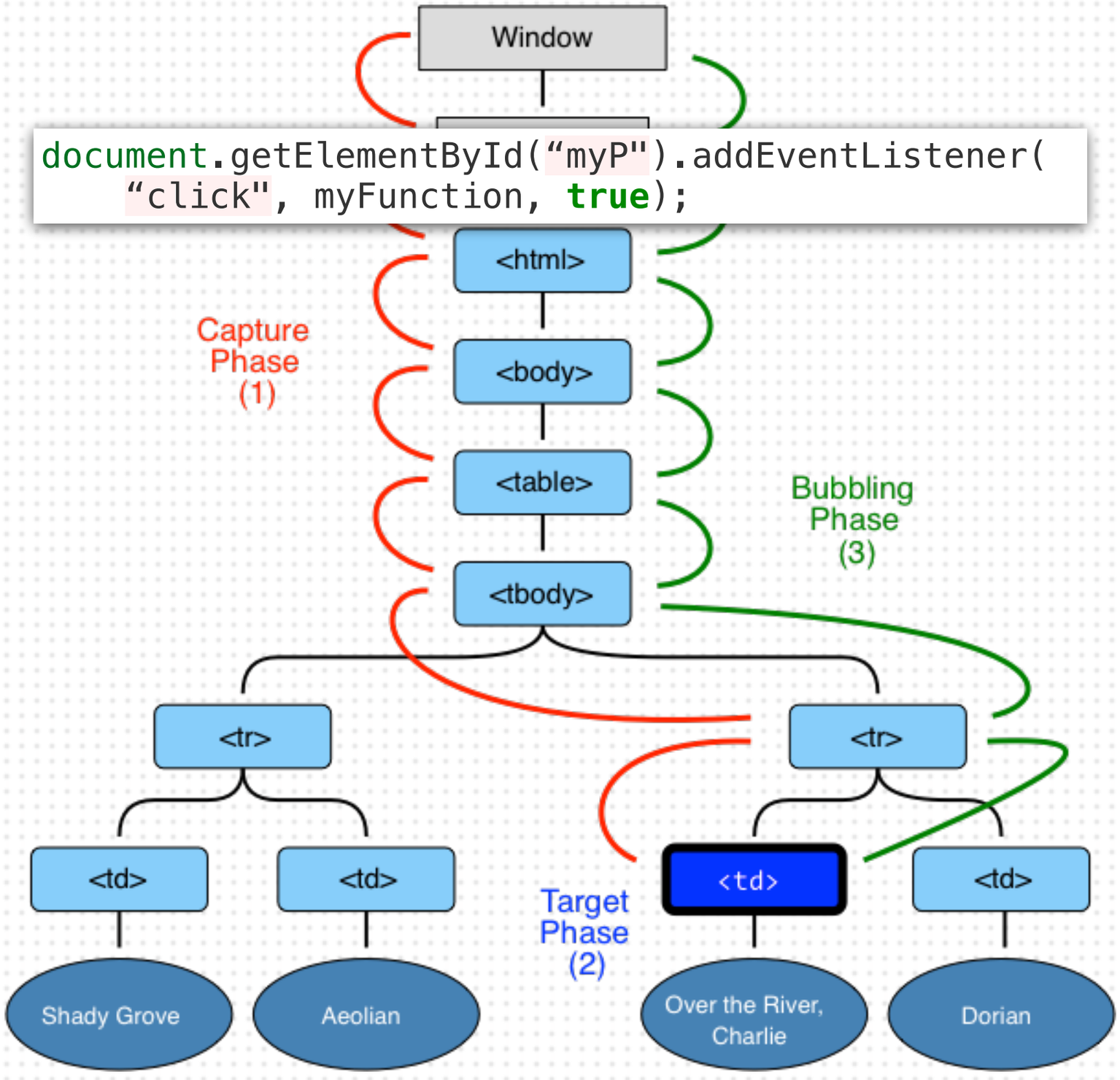
```
<html>
<body>
  <p id="p2">Hello World! </p>
  <script>
    document.getElementById("p2").style.color = "blue";
  </script>
  <p>The paragraph above was changed by a script. </p>
</body>
</html>
```

```
<htm1>
<body>
  <h1 id="id1">My Heading 1</h1>
  <button type="button"
    onclick="document.getElementById('idl').style.color = 'red'">
    Click Me!
  </button>
</body>
</html>
```

Events Bubbling and Capturing

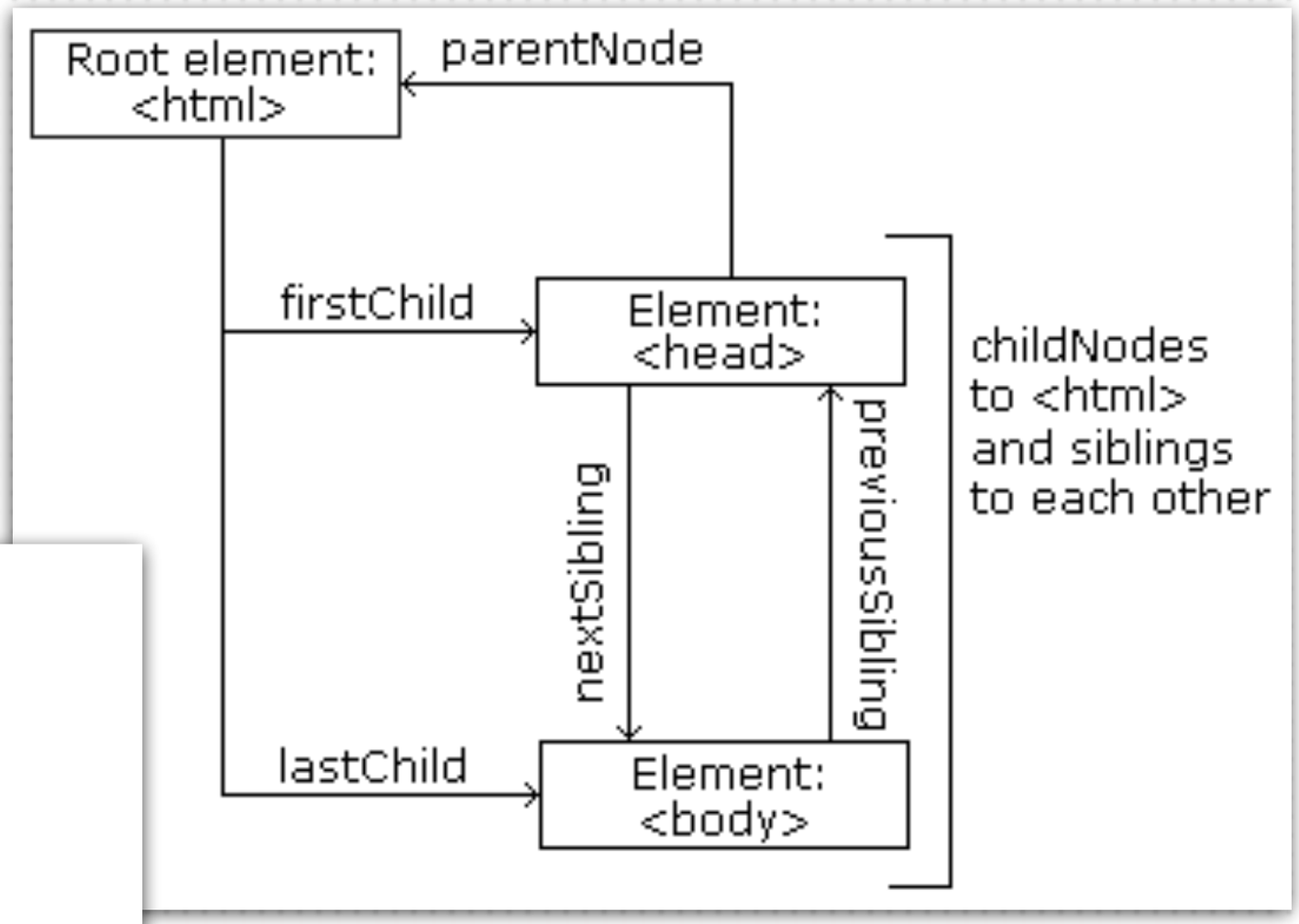


Events Bubbling and Capturing



Navigating DOM

- parentNode
- childNodes[nodenumber]
- firstChild
- lastChild
- nextSibling
- previousSibling




```
<html>
<head>
  <title>DOM Tutorial‹/title>
</head>
<body>
  <h1>DOM Lesson one</h1>
  <p>Hello world! </p>
</body>
</html>
```

Manipulating DOM

```
var nextId = 0;
function addInput) {
  var newDiv = document.createElement('div');
  newDiv.id = "choiceDiv_" + nextId;
  newDiv.innerHTML = "<input type='text'..."
  document.getElementById('choicesDiv').appendChild(newDiv);
  nextId++;
}
```

```
var myNodelist = document.getElementsByTagName ("p");
var i;
for (i = 0; i < myNodelist.length; i++) {
  myNodelist [i].style.backgroundColor = "red";
}
```

Browser Object Model

- window object (the top-level element)
- window.document
- window.screen
- window.location
- window.history
- window.navigator
- window.open()
- window.close()
- window.moveTo()

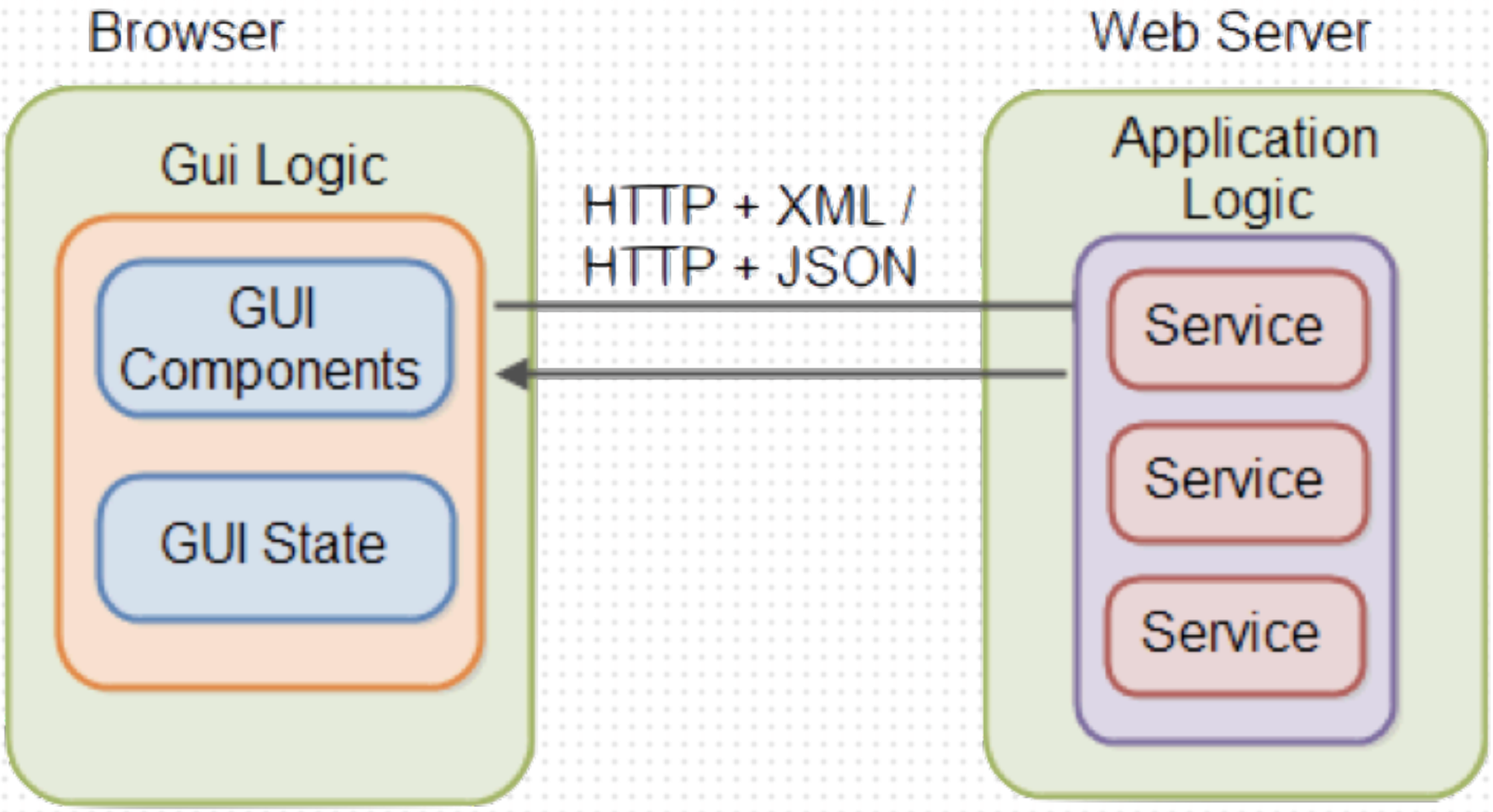
JavaScript Timing

```
<button onclick="t = setTimeout(myFunction, 3000)">
Try it
</button>

<script>
function myFunction() {
    alert('Hello');
}
</script>

<button onclick="clearTimeout(t)">Stop it</button>
```

Back to the RIA Web Apps



Data Exchange Formats

JSON: JavaScript Object Notation

```
{"employees":[
    {"firstName":"John", "lastName":"Doe"},
    {"firstName":"Anna", "lastName":"Smith"},
    {"firstName":"Peter", "lastName":"Jones"}
]}
```

```
<script>
  var text = '{"name": "John Johnson", "street": "Oslo West 16",
  "phone": "555 1234567"}';
  var obj = JSON.parse(text);
  document.getElementById("demo").innerHTML = obj name + "<br>" +
  obj.street + "<br>" + obj-phone;
</script>
```

AJAX (Asynchronous JavaScript and XML)

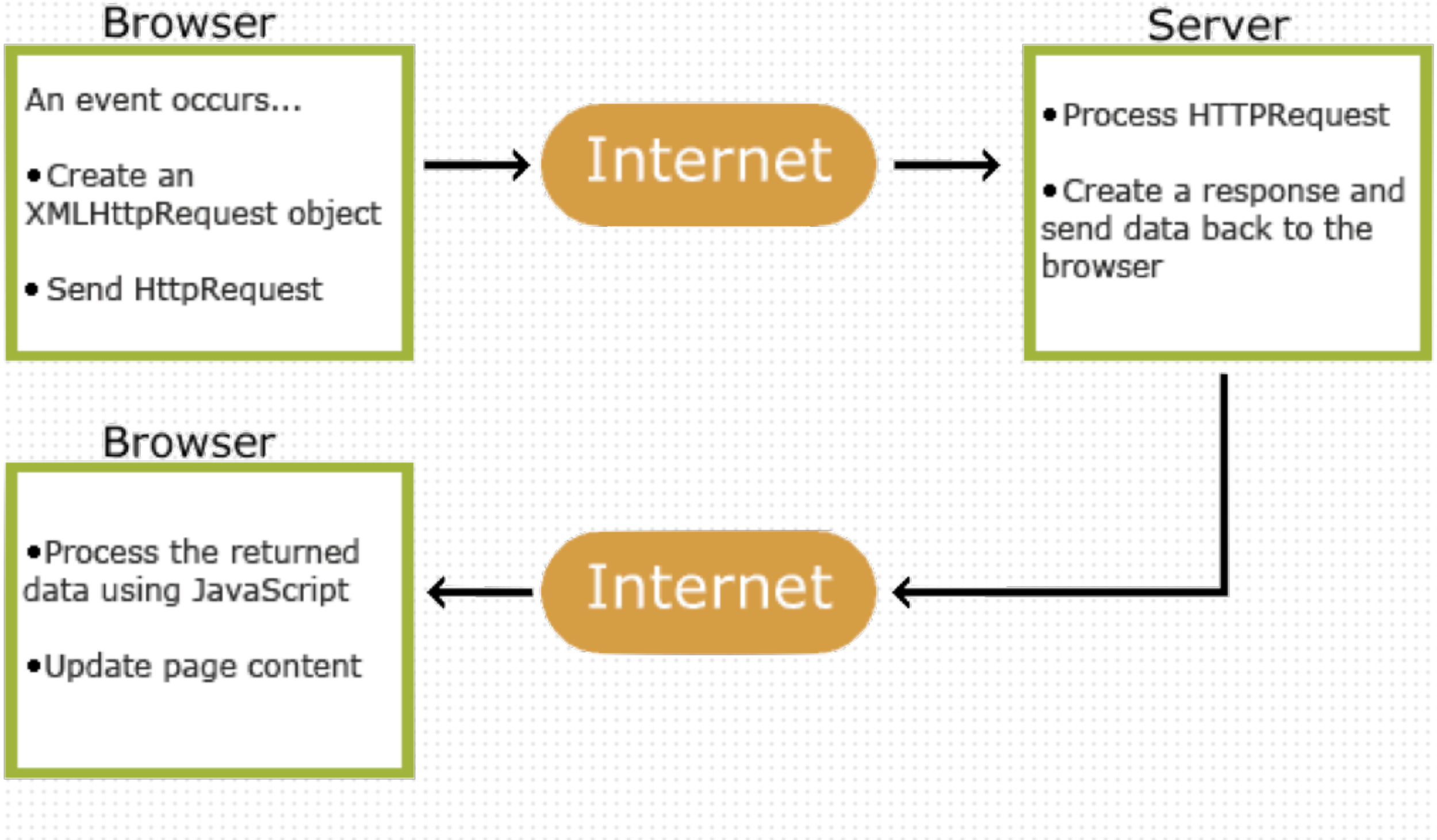
AJAX is a developers dream, because you can:

- Update a web page without reloading the page
- Request data from a server - after the page has loaded
- Receive data from a server - after the page has loaded
- Send data to a server - in the background

AJAX Example

```
<!DOCTYPE HTML>
<html>
<body>
  <script>
    function loadDoc() {
      var xhttp = new XMLHttpRequest ();
      xhttp.onreadystatechange = function () {
        if (xhttp.readyState == 4 && xhttp.status == 200) {
          document.getElementById(“demo").innerHTML = xhttp.responseText;
        }
      };
      xhttp.open("GET", "ajax_info.txt", true);
      xhttp.send ();
    }
  </script>
  <div id="demo"><h2>let AJAX change this text</h2></div>
  <button type="button" onclick="loadDoc()">Change Content</button>
</body>
```

How AJAX Works



The AJAX Request

```
xhttp.open("GET", "ajax_info.txt", true);
xhttp.send();
```

Method	Description
open(*method, url, async*)	Specifies the type of request *method*: the type of request: GET or POST *url*: the server (file) location *async*: true (asynchronous) or false (synchronous)
send()	Sends the request to the server (used for GET)
send(*string*)	Sends the request to the server (used for POST)

More About Ajax

- GET is simpler and faster than POST, and can be used in most cases; however, always use POST when:
  - Update a file or database on the server
  - Sending a large amount of data to the server
  - Sending user input (contains unknown characters)

```
xhttp.open("POST", “ajax_test.asp", true);
xhttp.setRequestHeader("Content-type",
   "application/x-www-form-urlencoded" );
xhttp.send("fname=Henry&1name=Ford");
```

- Synchronous Requests

```
http.open("GET", "ajax_info.txt", false);
xhttp.send();
document.getElementById("demo").innerHTML =
   xhttp.responseText;
```

onreadystatechange event

- Three important properties of the XMLHttpRequest object:

Property	Description
onreadystatechange	Stores a function (or the name of a function) to be called automatically each time the readyState property changes
readyState	Holds the status of the XMLHttpRequest. Changes from 0 to 4: 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
status	200: "OK" 404: Page not found

Demo

jQuery

- jQuery is a lightweight, "write less, do more", JavaScript library.
- Features for:
  - HTML/DOM manipulation
  - CSS manipulation
  - HTML event methods
  - Effects and animations
  - AJAX
  - Utilities

Using jQuery

- The jQuery library is a single JavaScript file

```
<head>
<script src="jquery-3.7.1.min.js"></script>
</head>
```

- If you don't want to download and host jQuery yourself, you can include it from a CDN
  - Google is an example of someone who host jQuery

```
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/
jquery.min.js"></script>
</head>
```

jQuery Syntax

- Basic syntax is: **$(selector).action()**
  - A $ sign to define/access jQuery
  - A (selector) to "query (or find)" HTML elements
  - A jQuery action() to be performed on the element(s)
- **$(this).hide()**- hides the current element
- **$("p").hide()**- hides all <p> elements
- **$(".test").hide()**- hides all elements with class="test"
- **$("#test").hide()**- hides the element with id="test"

jQuery Events

```
$("p").click(function(){
  $(this).hide ();
}) ;
```

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

TypeScript

- TypeScript = JavaScript + static types
- It compiles down to JavaScript (runs everywhere JS runs)
- Key idea: TypeScript catches errors before runtime
- Pros: fewer runtime bugs, better autocomplete, easier refactoring
- Cons: extra syntax, learning curve, build step required

TypeScript

- It prevents obvious bugs early

JavaScript

```
function greet(user) {
  return "Hello " + user.name;
}
greet(null); // runtime error 💥
```

TypeScript

```
function greet(user: { name: string }) {
  return "Hello " + user.name;
}
greet(null); // ❌ compile-time error
```

- Function Safety

JavaScript

```
function add(a, b) {
  return a + b;
}
add(2, "3"); // "23" 😬
```

TypeScript

```
function add(a:number, b:number): number
{
  return a + b;
}
add(2, "3"); // ❌ error
```

- Objects and APIs

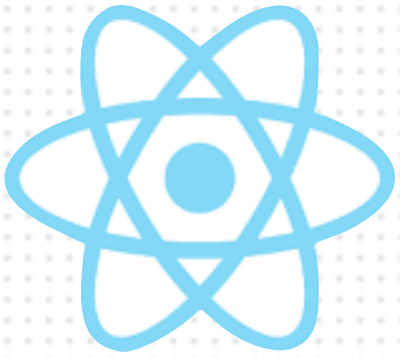
JavaScript

```
const user = { name: "Alex" };
console.log(user.age);
                // undefined 🤷‍♂️
```

TypeScript

```
type User = {
  name: string;
  age?: number;
};
const user: User = { name: "Alex" };
```

JavaScript Frameworks



React







Vue.js

